Oral Cancer Prediction – Top 30 Countries

Project Overview

Oral cancer is a major global health concern, with significant variations in incidence and survival rates across different countries. This project focuses on analyzing oral cancer prediction data for the top 30 countries based on prevalence and risk factors. The dataset includes demographic details, lifestyle habits, medical history, and genetic predisposition, providing valuable insights for early detection and prevention strategies.

Goals of the Project

Identify key risk factors contributing to oral cancer.

Develop predictive models to assess individual risk levels.

Compare trends across different countries to understand regional variations.

Apply machine learning techniques for classification and survival analysis.

Assist healthcare professionals in early diagnosis and personalized treatment planning.

Potential Use Cases

Exploratory Data Analysis (EDA): Uncover patterns and correlations in oral cancer risk factors.

Risk Prediction Models: Train ML models to estimate the likelihood of developing oral cancer.

Public Health Insights: Support policymakers in designing targeted awareness and prevention programs.

Medical Research: Enhance understanding of genetic and environmental influences on oral cancer.

In [12]:
```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
```

In [14]:
```
import warnings
warnings.filterwarnings("ignore")
```

In [17]:
```
df = pd.read_csv(r"C:\Users\chitt\Downloads\oral_cancer_prediction_dataset.csv")
```

In [19]:
```
df.head()
```

Out[19]:

	ID	Country	Gender	Age	Tobacco_Use	Alcohol_Use	Socioeconomic_Status	Diagnc
0	1	Ethiopia	Male	34	1	1	High	
1	2	Turkey	Female	84	1	1	High	
2	3	Turkey	Female	62	1	1	Middle	
3	4	Tanzania	Male	48	1	1	Middle	
4	5	France	Male	26	1	1	Middle	

In [21]:
```
df.tail()
```

Out[21]:

	ID	Country	Gender	Age	Tobacco_Use	Alcohol_Use	Socioeconomic_St
160287	160288	United Kingdom	Female	53	0	1	Mi
160288	160289	Brazil	Female	81	0	0	
160289	160290	Nigeria	Male	59	0	1	
160290	160291	Philippines	Female	43	0	0	
160291	160292	Nigeria	Female	20	0	0	

In [23]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 160292 entries, 0 to 160291
Data columns (total 11 columns):
 #   Column                Non-Null Count   Dtype  
---  ------                --------------   -----  
 0   ID                    160292 non-null  int64  
 1   Country               160292 non-null  object 
 2   Gender                160292 non-null  object 
 3   Age                   160292 non-null  int64  
 4   Tobacco_Use           160292 non-null  int64  
 5   Alcohol_Use           160292 non-null  int64  
 6   Socioeconomic_Status  160292 non-null  object 
 7   Diagnosis_Stage       160292 non-null  object 
 8   Treatment_Type        160292 non-null  object 
 9   Survival_Rate         160292 non-null  float64
 10  HPV_Related           160292 non-null  int64  
dtypes: float64(1), int64(5), object(5)
memory usage: 13.5+ MB
```

In [25]:
```
df.describe()
```

Out[25]:

	ID	Age	Tobacco_Use	Alcohol_Use	Survival_Rate	HI
count	160292.000000	160292.000000	160292.000000	160292.000000	160292.000000	160
mean	80146.500000	46.564102	0.601677	0.499638	0.599990	
std	46272.459012	20.594431	0.489554	0.500001	0.172882	
min	1.000000	20.000000	0.000000	0.000000	0.300002	
25%	40073.750000	29.000000	0.000000	0.000000	0.450680	
50%	80146.500000	39.000000	1.000000	0.000000	0.599586	
75%	120219.250000	64.000000	1.000000	1.000000	0.749291	
max	160292.000000	89.000000	1.000000	1.000000	0.899992	

In [27]:
```
df.dtypes
```

Out[27]:
```
ID                      int64
Country                object
Gender                 object
Age                     int64
Tobacco_Use             int64
Alcohol_Use             int64
Socioeconomic_Status   object
Diagnosis_Stage        object
Treatment_Type         object
Survival_Rate         float64
HPV_Related             int64
dtype: object
```

In [29]:
```
df.shape
```

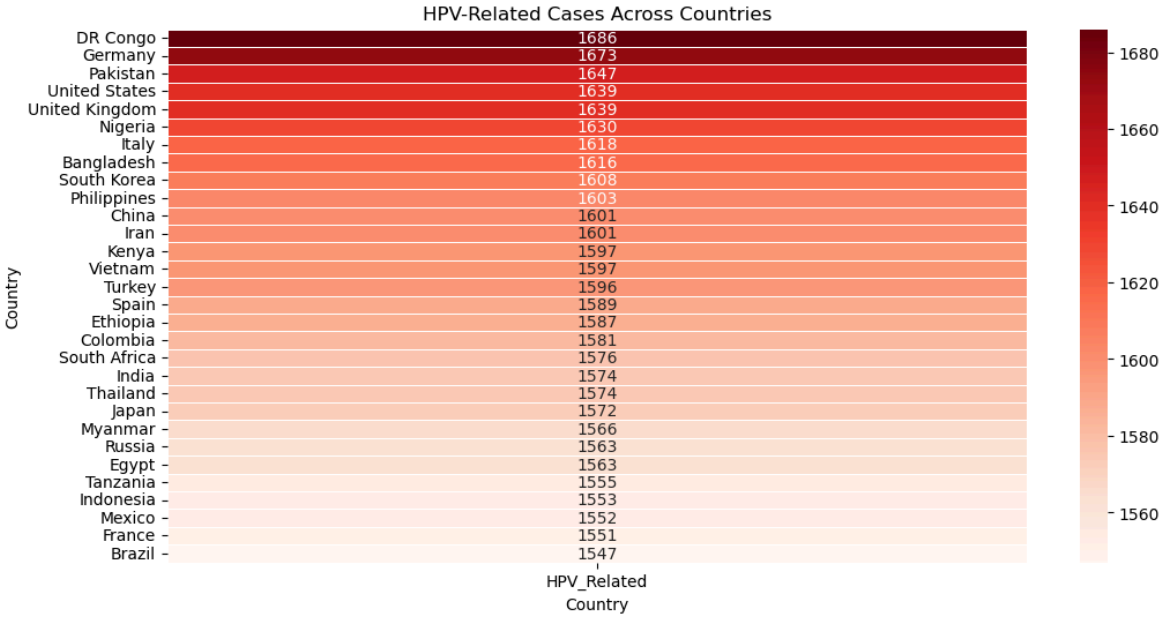
Out[29]:
```
(160292, 11)
```

In [31]:
```
df.columns
```

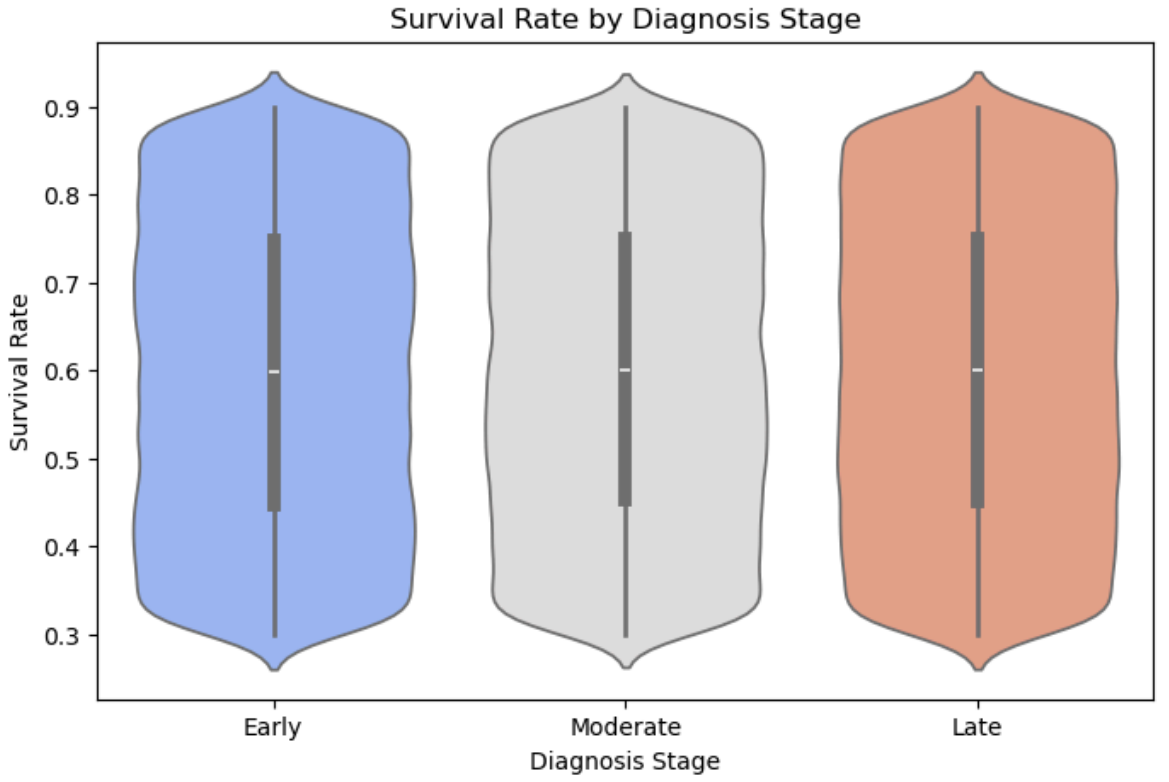
Out[31]:
```
Index(['ID', 'Country', 'Gender', 'Age', 'Tobacco_Use', 'Alcohol_Use',
       'Socioeconomic_Status', 'Diagnosis_Stage', 'Treatment_Type',
       'Survival_Rate', 'HPV_Related'],
      dtype='object')
```

EDA

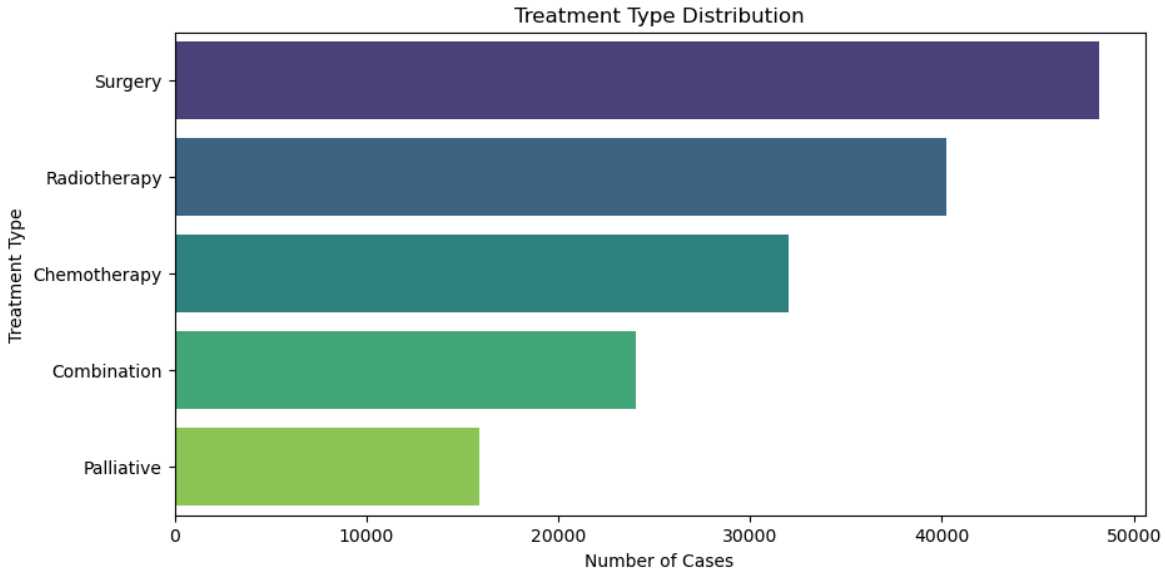
In [38]:
```
plt.figure(figsize=(12, 6))
hpv_counts = df.groupby("Country")["HPV_Related"].sum().reset_index()
hpv_counts = hpv_counts.sort_values(by="HPV_Related", ascending=False)
sns.heatmap(hpv_counts.set_index("Country"), annot=True, cmap="Reds", linewidths
plt.title("HPV-Related Cases Across Countries")
plt.xlabel("Country")
plt.show()
```



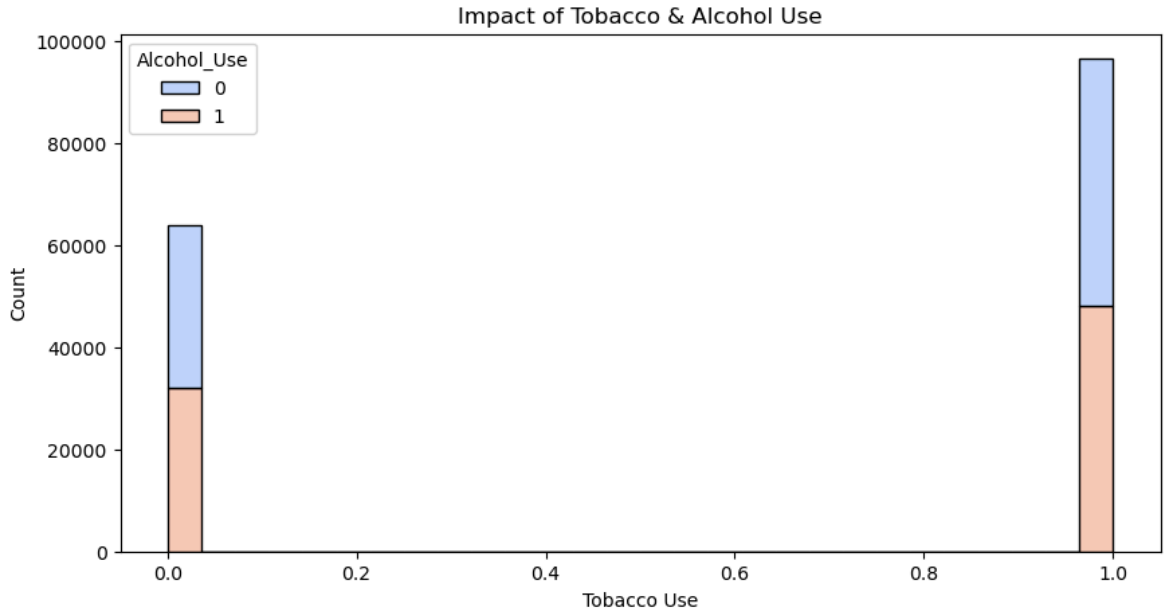
In [40]:
```
plt.figure(figsize=(8, 5))
sns.violinplot(x="Diagnosis_Stage", y="Survival_Rate", data=df, palette="coolwar
plt.xlabel("Diagnosis Stage")
plt.ylabel("Survival Rate")
plt.title("Survival Rate by Diagnosis Stage")
plt.show()
```



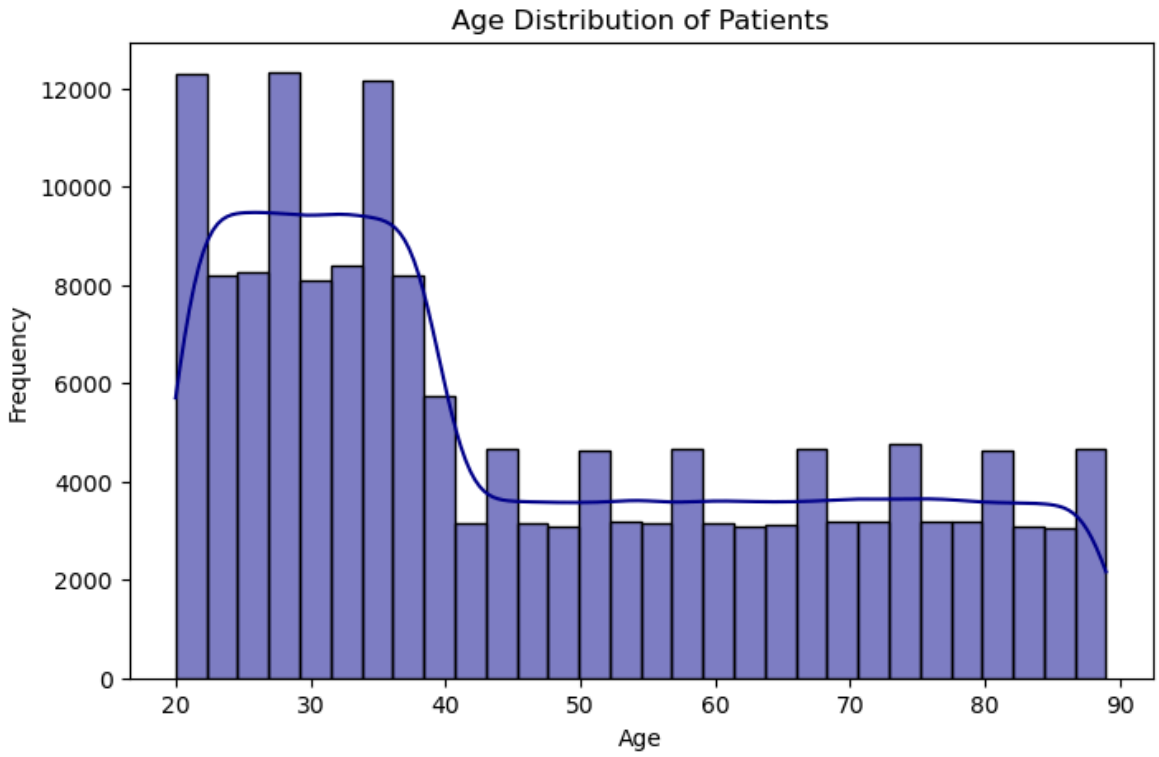
In [42]:
```
plt.figure(figsize=(10, 5))
sns.countplot(y=df['Treatment_Type'], order=df['Treatment_Type'].value_counts().
plt.xlabel("Number of Cases")
plt.ylabel("Treatment Type")
plt.title("Treatment Type Distribution")
plt.show()
```



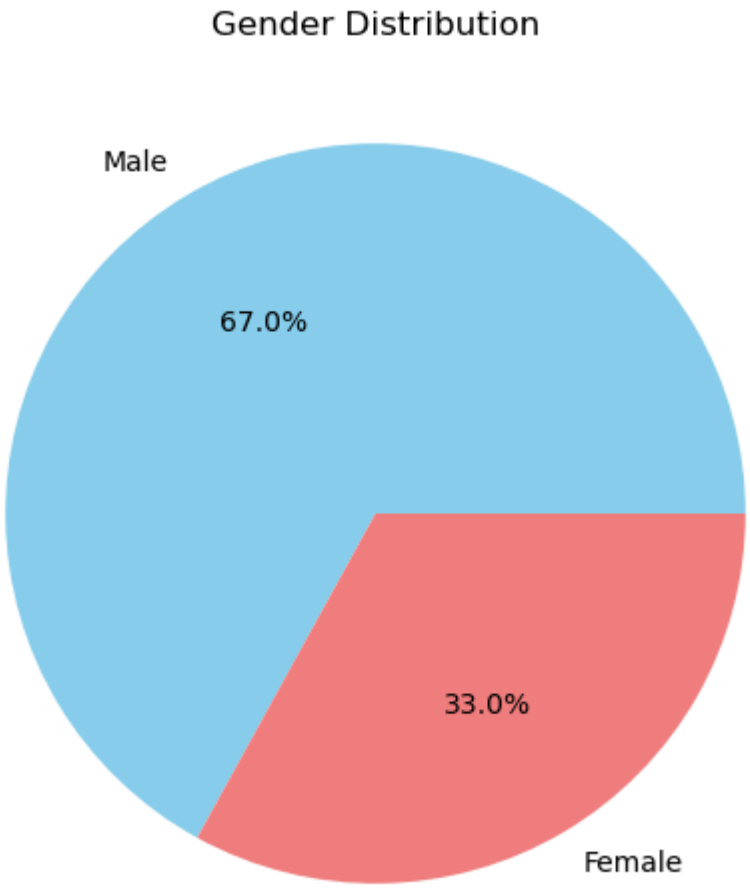
In [44]:
```
plt.figure(figsize=(10, 5))
sns.histplot(data=df, x="Tobacco_Use", hue="Alcohol_Use", multiple="stack", pale
plt.xlabel("Tobacco Use")
plt.ylabel("Count")
plt.title("Impact of Tobacco & Alcohol Use")
plt.show()
```



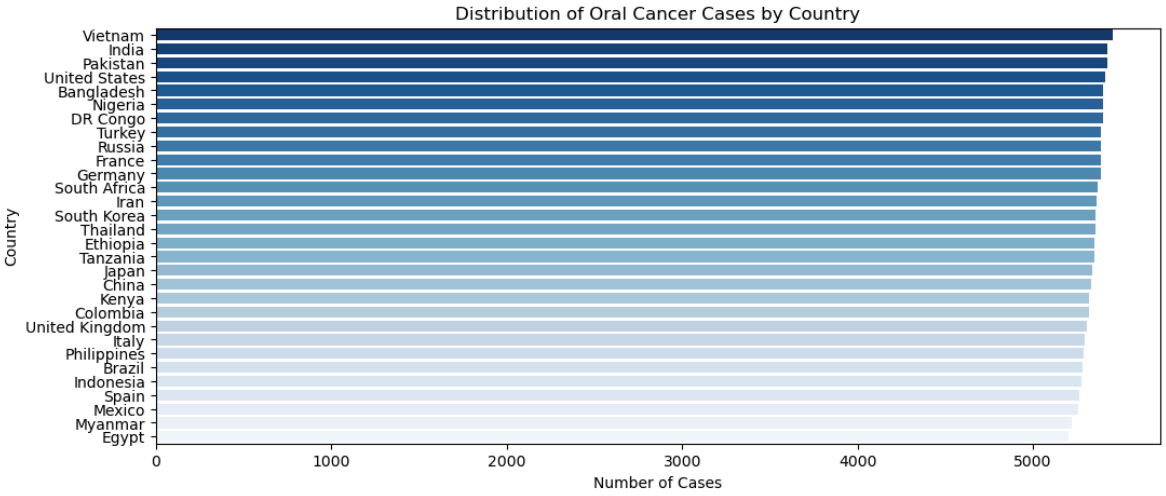
In [46]:
```
plt.figure(figsize=(8, 5))
sns.histplot(df['Age'], bins=30, kde=True, color="darkblue")
plt.xlabel("Age")
plt.ylabel("Frequency")
plt.title("Age Distribution of Patients")
plt.show()
```



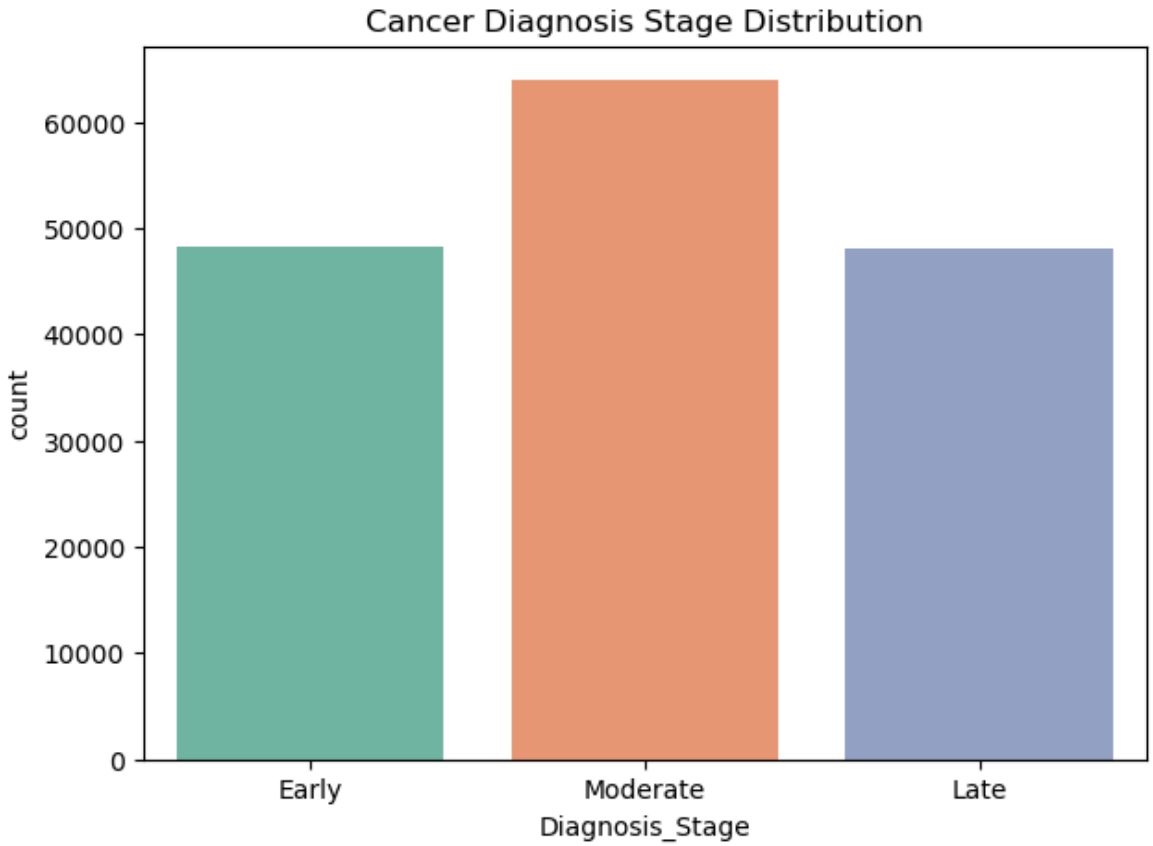
In [48]:
```
plt.figure(figsize=(6, 6))
df['Gender'].value_counts().plot.pie(autopct="%1.1f%%", colors=["skyblue", "ligh
plt.title("Gender Distribution")
plt.ylabel("")
plt.show()
```



In [50]:
```
plt.figure(figsize=(12, 5))
sns.countplot(y=df['Country'], order=df['Country'].value_counts().index, palette
plt.xlabel("Number of Cases")
plt.ylabel("Country")
plt.title("Distribution of Oral Cancer Cases by Country")
plt.show()
```

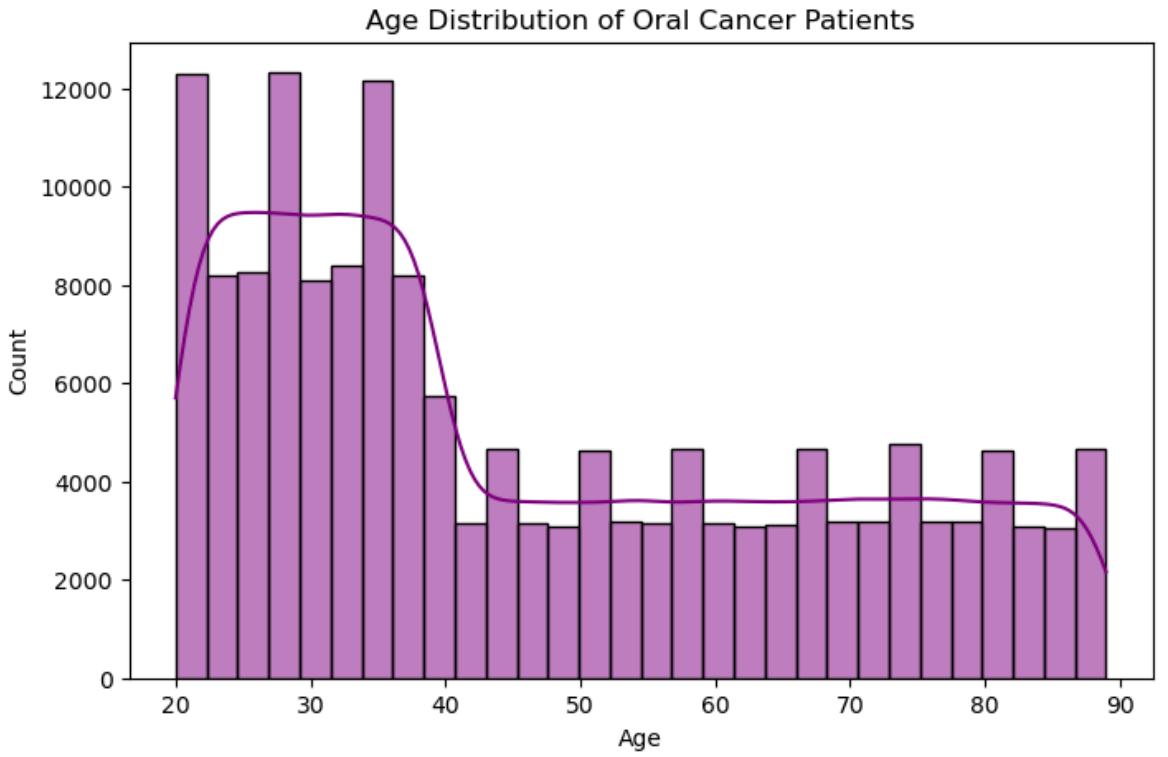


In [52]:
```
plt.figure(figsize=(7, 5))
sns.countplot(x="Diagnosis_Stage", data=df, order=["Early", "Moderate", "Late"],
plt.title("Cancer Diagnosis Stage Distribution")
plt.show()
```

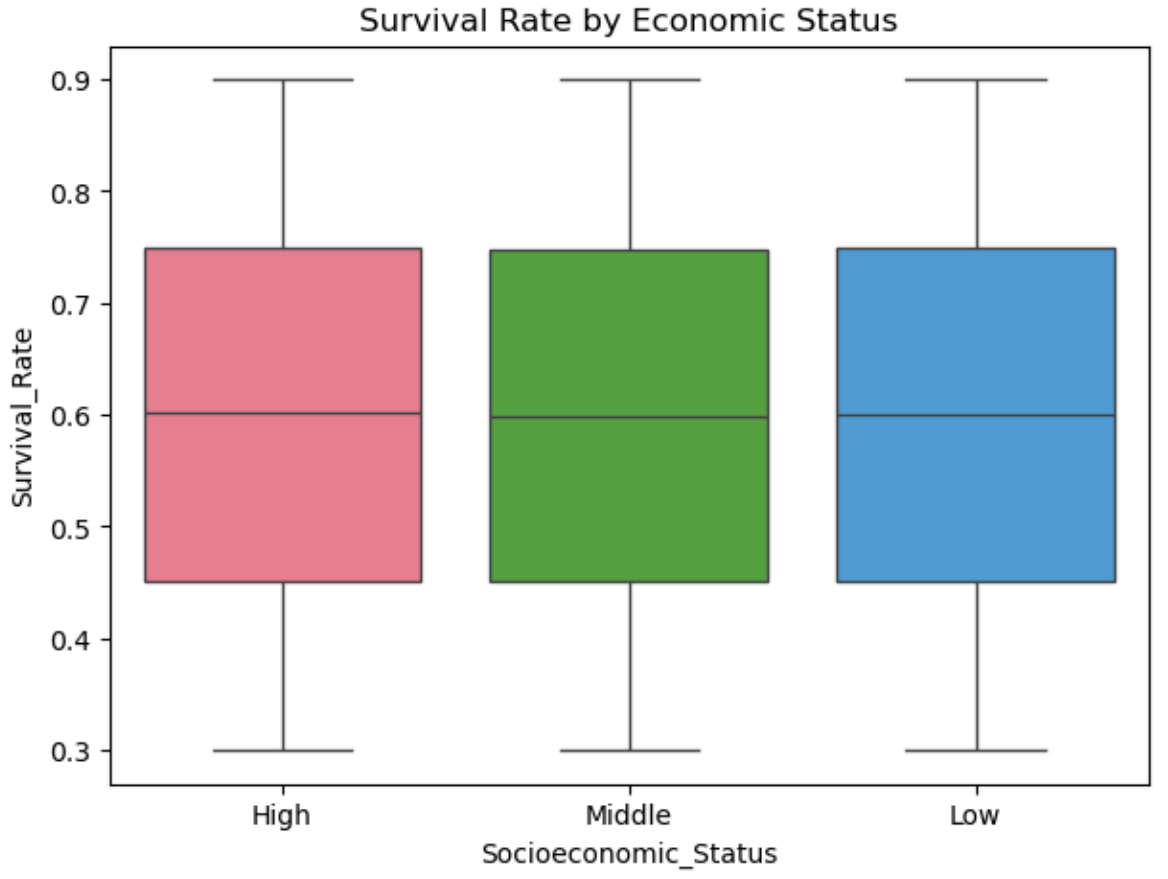


In [54]:
```
plt.figure(figsize=(8, 5))
sns.histplot(df["Age"], bins=30, kde=True, color="purple")
plt.title("Age Distribution of Oral Cancer Patients")
plt.xlabel("Age")
```

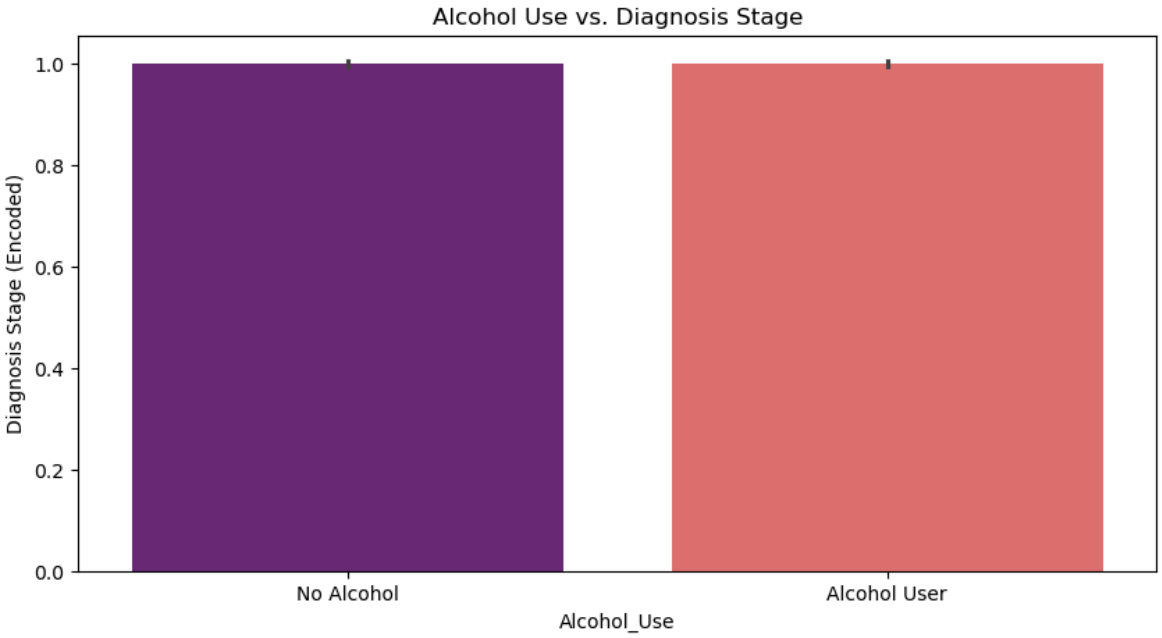
```
plt.ylabel("Count")
plt.show()
```



In [56]:
```
plt.figure(figsize=(7, 5))
sns.boxplot(x="Socioeconomic_Status", y="Survival_Rate", data=df, palette="husl"
plt.title("Survival Rate by Economic Status")
plt.show()
```



In [58]:
```
plt.figure(figsize=(10, 5))
sns.barplot(x="Alcohol_Use", y=df["Diagnosis_Stage"].factorize()[0], data=df, pa
plt.xticks([0, 1], ["No Alcohol", "Alcohol User"])
plt.ylabel("Diagnosis Stage (Encoded)")
plt.title("Alcohol Use vs. Diagnosis Stage")
plt.show()
```



Machine Learning

In [61]:
```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.linear_model import LinearRegression
from sklearn.svm import SVR
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
```

In [63]:
```
# Drop ID column if present
df = df.drop(columns=["ID"], errors='ignore')
```

In [65]:
```
# Encode categorical variables
label_encoders = {}
categorical_columns = ["Country", "Gender", "Socioeconomic_Status", "Diagnosis_S
for col in categorical_columns:
    le = LabelEncoder()
    df[col] = le.fit_transform(df[col])
    label_encoders[col] = le
```

In [67]:
```
# Define features and target
X = df.drop(columns=["Survival_Rate"])
y = df["Survival_Rate"]
```

In [69]:
```
# Reduce dataset size for efficient training
df_sampled = df.sample(n=5000, random_state=42)
X_sampled = df_sampled.drop(columns=["Survival_Rate"])
y_sampled = df_sampled["Survival_Rate"]
```

In [71]:
```
# Split the dataset
X_train, X_test, y_train, y_test = train_test_split(X_sampled, y_sampled, test_s
```

In [73]:
```
# Scale numerical features
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

In [77]:
```
# Define models
models = {
    "Linear Regression": LinearRegression(),
    "Random Forest": RandomForestRegressor(n_estimators=100, random_state=42),
    "Gradient Boosting": GradientBoostingRegressor(n_estimators=100, random_stat
    "Support Vector Regressor": SVR(kernel='rbf')
}
```

In [79]:
```
# Train and evaluate models
results = {}
for name, model in models.items():
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)

    mae = mean_absolute_error(y_test, y_pred)
    mse = mean_squared_error(y_test, y_pred)
    r2 = r2_score(y_test, y_pred)

    results[name] = {"MAE": mae, "MSE": mse, "R2 Score": r2}
```

In [81]:
```
# Display results
results_df = pd.DataFrame(results).T
print(results_df)
```

```
                            MAE       MSE  R2 Score
Linear Regression      0.150243  0.030164  0.003898
Random Forest          0.154154  0.032435 -0.071121
Gradient Boosting      0.151670  0.030850 -0.018758
Support Vector Regressor  0.155331  0.033458 -0.104879
```

Completed

In [ ]:
```

```